

Lumina NetOS

Security & Firewall Hardening Guide

Operational security guidance for deploying and running the discovery engine, licensing portal and appliance. Applies to v1.6.x.

1. Information-Security Precautions

Lumina NetOS holds privileged access to your estate (scan credentials, license keys, topology). Treat the host running the engine as a Tier-0 / management asset.

- **Protect the database.** The SQLite DB (%APPDATA%\LuminaNetOS\lumina.db, or /data in the container) stores LDAP config, the license, the admin password hash (PBKDF2-SHA256) and per-vendor credentials. Credentials are encrypted with **AES-256-GCM** using a per-install master key — but the DB file itself must be filesystem-protected and backed up securely.
- **Protect the signing key.** The portal's **Ed25519** license private key lives in portal.db (meta table). Anyone with it can forge licenses — restrict access and back it up offline.
- **Prefer SNMPv3 (NoAuthNoPriv) over v2c.** The engine tries v3 first and falls back to v2c. Avoid v2c where possible; never leave a public/private community in production.
- **Disable cleartext management.** Turn off Telnet (23) and HTTP-only admin on devices; the Security AI flags these. Use SSH and HTTPS.
- **Keep the engine API private.** The core binds 127.0.0.1:8080 by default. Only the appliance/Docker build binds 0.0.0.0:8080 — put it behind TLS and an admin-only firewall; **never expose :8080 to the internet.**
- **Authentication.** Use real **LDAP** auth + per-user **TOTP MFA**; the built-in local admin is restricted to Settings in production.
- **Gate downloads.** The public site only reveals the app download after registration/login — keep it that way.

2. Firewall Ports

Inbound — the application's own listeners (restrict to localhost / a dedicated admin VLAN):

Service	Port	Direction	Notes
Core engine (API+UI)	8080/tcp	Loopback*	Desktop: 127.0.0.1 only. Appliance/Docker: 0.0.0.0 — admin VLAN + TLS.
Licensing portal	8090/tcp	Inbound	Public landing/signup/releases. Put a reverse proxy + TLS in front.
Vite dev server	3000/tcp	Loopback	Development only — never exposed in production.

Outbound — what the discovery scanner needs to reach managed devices (open only from the scanner host to the device/management subnets):

Protocol	Port	Direction	Used for
SSH	22/tcp	Outbound	Network gear, Linux/Unix, ESXi, Nutanix, NAS, NetScaler.
Telnet	23/tcp	Outbound	Legacy only — flagged as a risk; prefer disabling on devices.
HTTP	80/tcp	Outbound	Web fingerprint / redirect check (plaintext is flagged).
HTTPS	443/tcp	Outbound	Web UI / cert fingerprint for firewalls, ADCs, appliances.
SMB	445/tcp	Outbound	Windows host detection.
WinRM	5985/tcp	Outbound	Windows discovery (WMI/PowerShell over WinRM).
SNMP	161/udp	Outbound	Interfaces, identity, ARP, throughput (v3 preferred, v2c fallback).
RDP	3389/tcp	Outbound	Windows endpoint/role detection; one-click remote.
MikroTik Winbox	8291/tcp	Outbound	MikroTik RouterOS detection.
Alt-HTTPS	8443, 4434/tcp	Outbound	Vendor admin GUIs (e.g. pfSense, appliances).

Place the management plane on a **dedicated admin/OOB VLAN**. The scanner host should reach devices on the ports above through a tightly-scoped firewall rule — not a blanket any/any.

3. What to Be Careful About

- **Least-privilege scan accounts.** Use read-only / monitoring accounts per vendor. The engine only needs to read config and inventory — never give it write/enable privileges.
- **Don't scan from a user VLAN.** Run the engine from the admin VLAN. A scanner on a user segment both exposes credentials to that segment and is itself a finding (management reachable from users).
- **Rotate SNMP communities & credentials.** Change default/community strings, rotate periodically, and store them only in the engine (encrypted) — not in spreadsheets.
- **Keep the license key secret.** The portal-issued org license key activates production; treat it like a secret.
- **Patch the engine host & devices.** The Security AI surfaces CVE / CISA-KEV matches and stale Windows patches — act on criticals first.
- **Back up safely.** Back up the DB (WAL-checkpointed) to encrypted storage; it contains everything needed to impersonate the platform.